

Práctica 3:

Mostrar en RVIZ múltiples robots

Multirobots

Nicolás Fernández Blánquez
9-3-2025

1. CONFIGURACIÓN INICIAL

Para comenzar, se ha creado un archivo *.world* en el que se ha descrito la configuración del mundo de Gazebo. En este, se ha especificado las dimensiones de varios bloques que actúan como obstáculos.

Una vez configurado el entorno con el que se iba a trabajar, se han introducido tres robots en diferentes ubicaciones. Para esto, se han utilizado los archivos de lanzamiento *one_robot.launch*, *robots.launch* y *main.launch* de prácticas anteriores.

Con esta configuración inicial, se ha ejecutado el mundo en gazebo para verificar su funcionamiento y se ha obtenido el siguiente resultado:

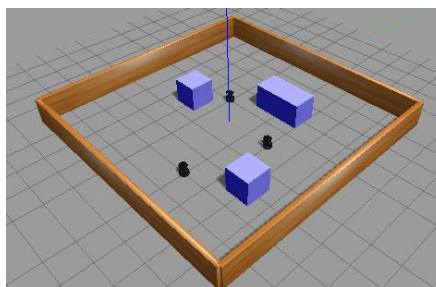


Imagen 1. Entorno en Gazebo

A continuación, se ha comprobado la estructura del árbol de transformaciones utilizando la herramienta *rqt_tf_tree*. Para ello, se ha ejecutado el siguiente comando:

```
roslaunch rqt_tf_tree rqt_tf_tree
```

Con esto, se ha podido ver el árbol de transformaciones y se ha observado que cada robot generaba su propio árbol junto con los componentes de su estructura. Es por ello por lo que ha sido necesario hacer una transformada para relacionar cada uno de los robots con el mapa. Para ello, se ha editado el fichero *robots.launch*:

```
<!-- BEGIN ROBOT 1 -->
<group ns="robot1">
  <param name="tf_prefix" value="robot1" />
  <include file="$(find practica03)/launchs/one_robot.launch" >
    <arg name="init_pose" value="-x -1 -y 1 -z 0 -R 0.00 -P 0.00 -Y 1.50" />
    <arg name="robot_name" value="robot1" />
  </include>
  <node pkg="tf" type="static_transform_publisher" name="map_robot1"
    args="-1 1 0 0 0 1.50 1 map robot1/odom 100" />
</group>
```

Imagen 2. Transformada entre el mapa y el robot

Esta nueva línea, añadida a cada uno de los robots, hace posible la transformada entre *map* y *robotX/odom*, es decir, se ha fijado la posición y orientación de cada uno de los robots dentro del frame “map”. [1]

En la imagen adjunta al paquete de ros y este informe, *tree.png*, se puede ver el árbol de transformaciones resultante.

Una vez se ha tenido la relación entre cada uno de los robots y el mapa, se ha configurado un fichero de RViz (.rviz) con una configuración predeterminada y se ha asociado a un fichero *.launch* para poder ejecutarlo, este fichero es *rviz.launch*.

Para comprobar que la configuración ha sido correcta, se han añadido los modelos de los robots utilizando *RobotModel*. Con esto, ha sido necesario añadir el nombre de cada uno de los robots a la casilla *TF Prefix*, obteniendo lo siguiente:

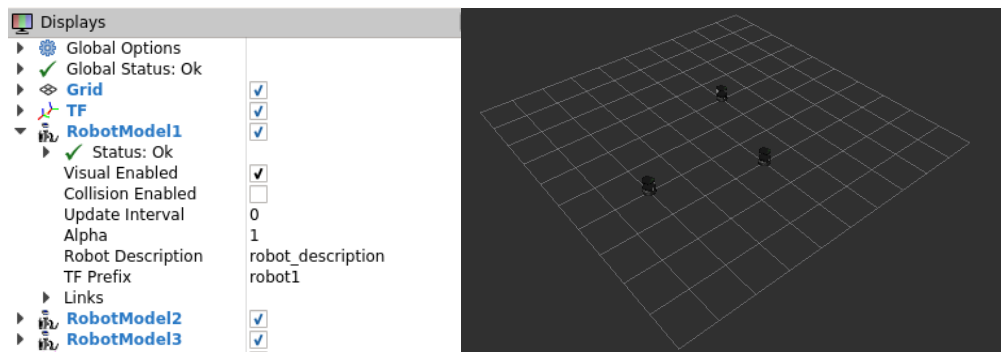


Imagen 3. Configuración de los modelos del robot y su visualización

Por último, para comprobar que las transformadas se estaban realizando correctamente, se ha añadido la pestaña TF:

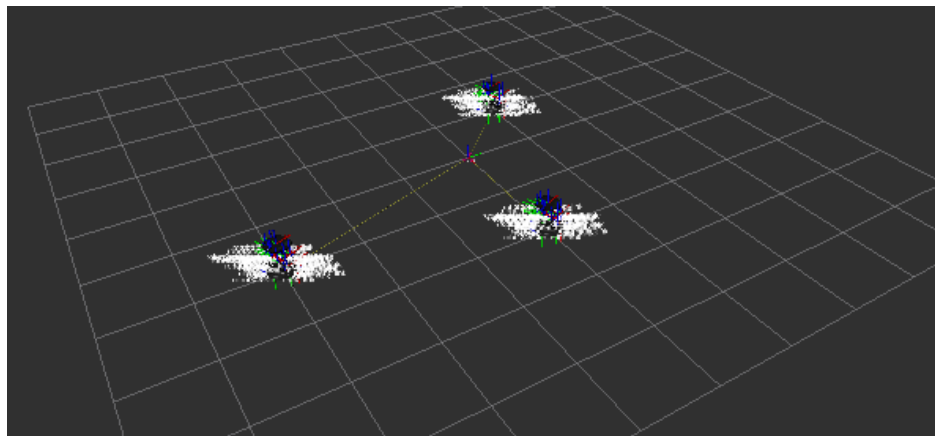


Imagen 4. TF entre los robots y el mapa

Por lo que se ha podido observar que se estaba relacionando de forma correcta las coordenadas de cada uno de los robots con el mapa.

2. ERRORES ENCONTRADOS

En cuanto a los errores encontrados, a la hora de mostrar el árbol de transformadas, aparecían los tres árboles principales con la estructura de los robots y luego, tres árboles de menor dimensión con el nombre de cada robot, pero en mayúscula.

Tras una larga investigación, se cayó en la cuenta de que en el fichero *robots.launch*, se escribía el nombre de cada uno de los robots de forma diferente. Para solucionarlo, se han escrito todos de la misma forma, es decir, en minúscula.

```
<!-- BEGIN ROBOT 1 -->
<group ns="robot1">
  <param name="tf_prefix" value="robot1" />
  <include file="$(find practica03)/launchs/one_robot.launch" >
    <arg name="init_pose" value="-x -1 -y 1 -z 0 -R 0.00 -P 0.00 -Y 1.50" />
    <arg name="robot_name" value="robot1" />
  </include>
  <node pkg="tf" type="static_transform_publisher" name="map_robot1"
    args="-1 1 0 0 0 1.50 1 map robot1/odom 100" />
</group>
```

Imagen 5. Corrección del fichero 'robots.launch'

Este mismo cambio se realizó en los 3 robots con los que se estaba trabajando, por lo que se solucionó el fallo y se pudo continuar.

3. PRUEBAS REALIZADAS

Una vez funcionaba todo a la perfección, se han probado los tópicos que se estaban publicando y comprobado la información que se obtenía de cada uno de ellos.

Como ya se ha comentado anteriormente, se han añadido los modelos de los robots y la pestaña TF.

Tras esto, se ha probado con el sensor IMU de los robots, este sensor se encarga de medir el movimiento del robot, es decir, la aceleración, velocidad angular y orientación.

Como todos los robots están girando sobre su propio eje Z, se ha visualizado una flecha morada en la dirección positiva de este eje.

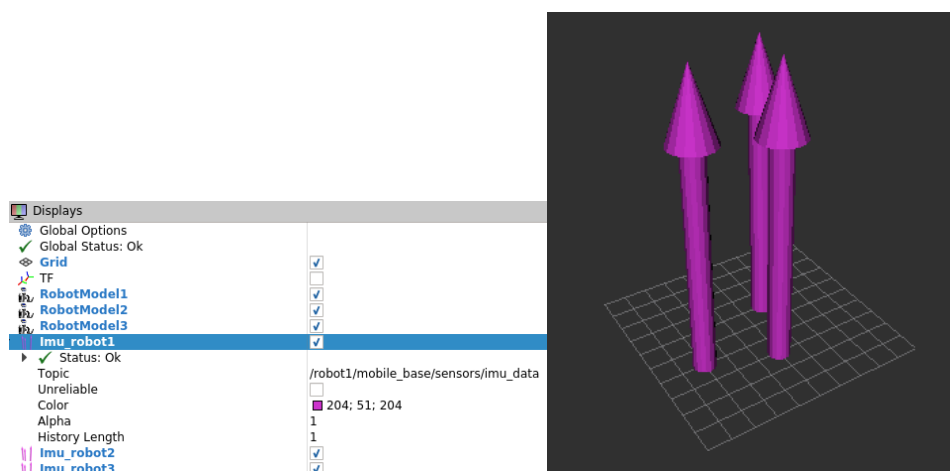


Imagen 6. Configuración del sensor IMU y su respectiva visualización

También, se ha añadido la pestaña *odometry* la cual muestra la estimación de la posición y orientación del robot en el espacio.

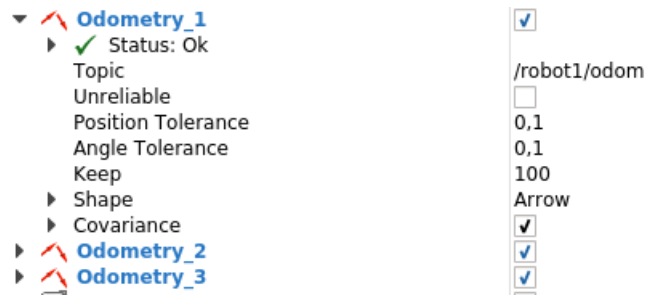


Imagen 7. Configuración de 'odometry'

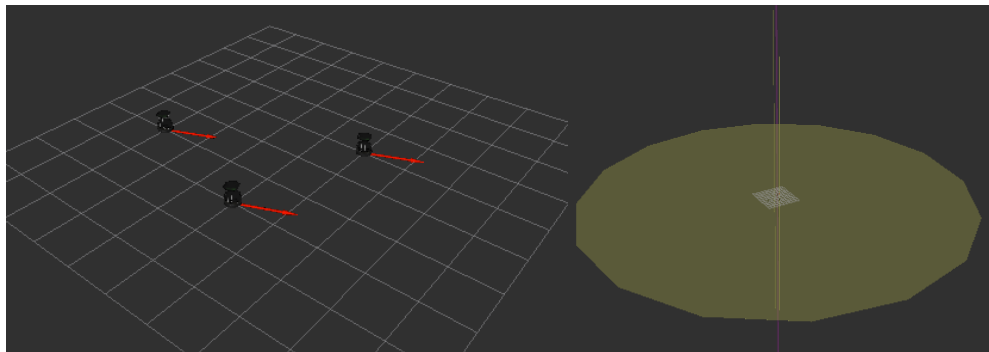


Imagen 8. Representación de la orientación

Imagen 9. Representación de la posición

En la imagen 8, se puede ver la orientación que toma cada uno de los robots, representada por una flecha de color rojo. Por otro lado, en la imagen 9 se puede ver el espacio de color amarillo que representa el área de incertidumbre en la estimación de la posición del propio robot.

Luego, se ha añadido la pestaña *LaserScan* y asociado el laser de cada uno de los robots, pero no se ha apreciado en RVIZ. Con esto, se ha comprobado si realmente estaba publicando información en ese nodo y se ha podido ver que no funcionaba.

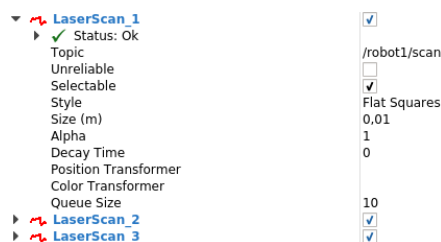


Imagen 10. Configuración de 'LaserScan'

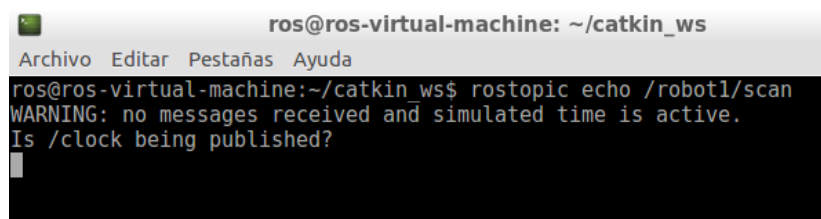
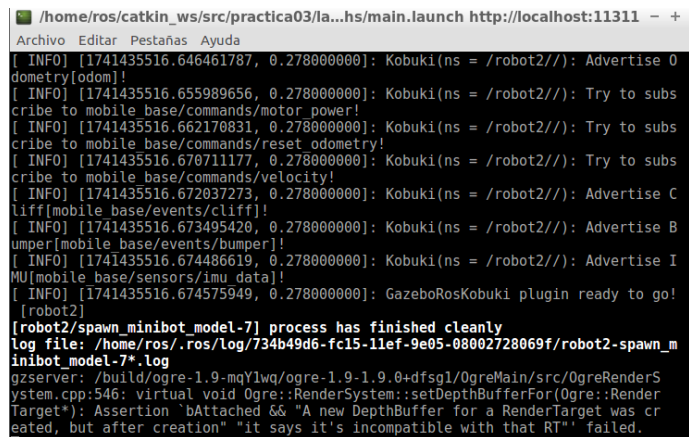


Imagen 11. Comprobación del funcionamiento

Por último, se ha intentado añadir la pestaña *Camera* o *Image* para poder visualizar la cámara del robot. Si bien es cierto que no se ha apreciado ningún error en RVIZ más que un *warning* que avisaba que no se estaban recibiendo imágenes, se ha podido ver en la terminal donde se ha lanzado gazebo lo siguiente:



```

/home/ros/catkin_ws/src/practica03/la...hs/main.launch http://localhost:11311 - +
Archivo Editar Pestañas Ayuda
[ INFO] [1741435516.646461787, 0.278000000]: Kobuki(ns = /robot2//): Advertise O
dometry[odom]!
[ INFO] [1741435516.655989656, 0.278000000]: Kobuki(ns = /robot2//): Try to subs
cribe to mobile_base/commands/motor_power!
[ INFO] [1741435516.662170831, 0.278000000]: Kobuki(ns = /robot2//): Try to subs
cribe to mobile_base/commands/reset_odometry!
[ INFO] [1741435516.670711177, 0.278000000]: Kobuki(ns = /robot2//): Try to subs
cribe to mobile_base/commands/velocity!
[ INFO] [1741435516.672037273, 0.278000000]: Kobuki(ns = /robot2//): Advertise C
liff[mobile_base/events/cliff]!
[ INFO] [1741435516.673495420, 0.278000000]: Kobuki(ns = /robot2//): Advertise B
umper[mobile_base/events/bumper]!
[ INFO] [1741435516.674486619, 0.278000000]: Kobuki(ns = /robot2//): Advertise I
MU[mobile_base/sensors/imu_data]!
[ INFO] [1741435516.674575949, 0.278000000]: GazeboRosKobuki plugin ready to go!
[robot2]
[robot2/spawn_minibot_model-7] process has finished cleanly
log file: /home/ros/.ros/log/734b49d6-fc15-11ef-9e05-08002728069f/robot2-spawn_m
inibot_model-7*.log
gzserver: /build/ogre-1.9-mqYlwg/ogre-1.9-1.9.0+dfsg1/OgreMain/src/OgreRenderS
ystem.cpp:546: virtual void Ogre::RenderSystem::setDepthBufferFor(Ogre::Render
Target*): Assertion 'bAttached && "A new DepthBuffer for a RenderTarget was cr
eated, but after creation" "it says it's incompatible with that RTTI" failed.

```

Imagen 12. Error en la terminal

Se ha investigado la causa de este error y se ha encontrado un repositorio en GitHub [\[2\]](#) donde se comenta que el error está relacionado con el sistema de renderizado *Ogre* en máquinas virtuales. Se comenta que es debido a incompatibilidades o limitaciones en la capacidad de renderizado 3D de la propia máquina. Si bien es cierto que se plantea una solución, no se ha querido comprobar su funcionamiento para evitar cualquier problema con la máquina virtual

4. RESULTADOS

En cuanto a los resultados obtenidos, se ha hecho girar a los 3 robots sobre su eje Z para ver la evolución del comportamiento de los sensores comentados anteriormente.

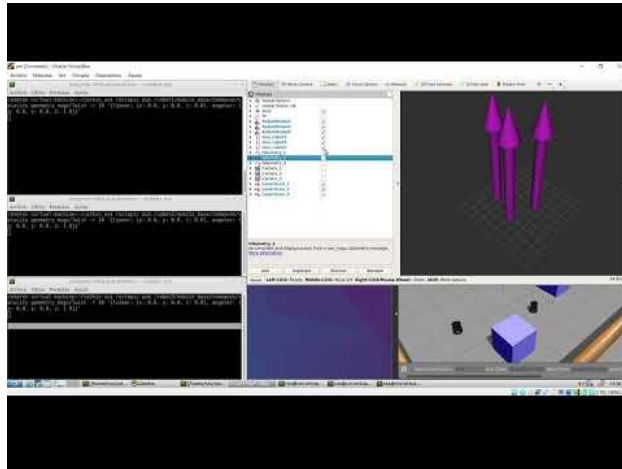
Para conseguir el movimiento, se ha simplificado publicando directamente la velocidad de giro del eje Z en el topic correspondiente utilizando los siguientes comandos:

```
rostopic pub /robot1/mobile_base/commands/velocity geometry_msgs/Twist -r 10
'{linear: {x: 0.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.0}}'
```

```
rostopic pub /robot2/mobile_base/commands/velocity geometry_msgs/Twist -r 10
'{linear: {x: 0.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.0}}'
```

```
rostopic pub /robot3/mobile_base/commands/velocity geometry_msgs/Twist -r 10
'{linear: {x: 0.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.0}}'
```

A continuación, se adjunta un video donde se puede ver el funcionamiento a tiempo real.



Video 1. Comprobación de los topics

5. BIBLIOGRAFÍA

1. Uzh-Rpg. (n.d.). *gazebo died when I try to get image_view · Issue #58 · uzh-rpg/rpg_quadrotor_control*. GitHub. https://github.com/uzh-rpg/rpg_quadrotor_control/issues/58
2. *tf - ROS Wiki*. (n.d.). https://wiki.ros.org/tf#static_transform_publisher